

Understanding and Applying DevOps Principles

COMP2013 - Lab 2

A practical transformation plan

Team details

Team members

Qixuan Yang (20719443)

Ronghao Hu (20719451)

Hanbo Zhao (20719456)

Team name / number

Not provided - complete before submission

Background of the organization

People and products

About 100 people at one site; three development teams of 10-15 people. Three web applications share a common database.

Current delivery process

Sales gathers requirements; teams use their own processes and tools. Development performs functional tests; Test performs manual integration and acceptance tests; Operations deploys to Google Cloud.

Communication

Informal email is the primary channel. The business domain and current delivery performance are not specified.

Task 1 - Challenge register

ID / challenge	Case evidence	Potential consequence	Plan response
C1 Communication	Informal email; sales collects requirements	Lost decisions, slow clarification	P1 Shared backlog and pilot squad
C2 Integration risk	Separate team tools; shared database	Conflicting changes, rework	P2 Shared checks; P3 migration controls
C3 Release bottleneck	Manual tests; Test-to-Ops handoff	Queues and deployment errors	P2 Automation; P3 safe releases
C4 Learning and trust	Separated departments; learning culture unknown	Knowledge silos if incentives conflict	P4 Feedback; L1-L3 learning actions

Consequences are reasoned possibilities. The assumptions behind them are stated next.

Task 1 - Communication and trust

C1: assumptions

A1: there is no shared, authoritative backlog or decision log. A2: important release decisions are held in individual inboxes.

C1: implications

Teams may implement different priorities or lose context at handoffs. A shared backlog, named product owner and documented release decisions would reduce ambiguity.

C4: assumption and implication

A3: Dev is rewarded for features and Ops for uptime. If true, conflicting incentives discourage risk reporting and sharing. Validate through interviews; introduce shared outcomes and no-blame reviews.

Task 1 - Tools and shared data

C2: assumptions

A4: teams have incompatible build or release conventions. A5: one application can introduce database changes that affect the other two.

Implications

Integration defects may appear late; a database change can block several releases. Tool differences may duplicate work and make troubleshooting harder.

Response and trade-off

Agree minimum build/test checks and versioned migration rules. Test all three applications against changed schemas. Keep useful team tools; defer a full database split because cost and feasibility are unknown.

Task 1 - Testing and release queues

C3: assumptions

A6: the Test team has limited capacity and repeated regression work. A7: deployment depends on manual steps and person-specific knowledge.

Implications

Work waits between teams; large release batches increase rework and recovery effort. Faster development alone can enlarge the test queue.

Response and trade-off

Measure queue time first. Automate critical regression checks, retain exploratory acceptance tests, and limit work in progress. Give the pilot squad joint responsibility for release readiness.

Task 2 - Objectives and approach

First Way: flow across the whole system

Make work visible, limit work in progress, reduce batch sizes and handoffs, then improve the measured constraint.

Proposed 12-week outcomes

Pilot: halve median commit-to-production time; deploy at least weekly; do not worsen the change failure rate; demonstrate rollback within 10 minutes. Baselines are unknown until weeks 1-2.

Pilot before scaling

Select one application and one critical user journey. Continue normal development; reserve pilot capacity rather than stop the whole company. Broader rollout depends on the pilot evidence.

Task 2 - P1: Workflow and ownership

When / owner	Action	Output / acceptance
Weeks 1-2 Transformation lead + Sales	Interview all roles; map demand to production; nominate a Sales product owner	Value-stream map; agreed priorities; named owners; baseline queue times
Week 2 Dev + Test + Ops leads	Create a pilot squad: 2 developers, 1 tester, 1 Ops engineer; Sales owns priorities	Squad charter; shared definition of done; release responsibility accepted
From week 2 Pilot squad	Use one Kanban board; start with WIP ≤ 3 active changes; unblock work daily	Each change has owner and acceptance criteria; weekly review of aging work

Suggested tools: a shared issue board and wiki. Retain existing tools if they satisfy the requirements.

Task 2 - P2: Build and test pipeline

Weeks 3-5 | Dev lead + Test lead

Version application code, pipeline files and deployment scripts. Add peer review and required checks; integrate small changes daily. Proposed pilot tools: Git and GitHub Actions.

Executable path

Commit -> build -> unit tests -> integration tests against a test database -> acceptance checks -> versioned artifact -> staging. Failed checks block release and return to the change owner.

Acceptance and documents

The pilot critical journey passes automatically in staging. Test all three applications when shared schemas change. Publish pipeline setup, test ownership and a release checklist; retain exploratory acceptance testing.

Task 2 - P3: Safe cloud releases

Weeks 4-6 | Ops lead + database owner

Create a reproducible staging environment and versioned infrastructure configuration; propose Terraform on existing Google Cloud. Add versioned migrations and compatibility checks for all three applications.

Production release control

Promote the tested artifact after a named Ops approver signs the checklist. Release one small change at a time; observe errors and latency. Revert the application if the agreed guardrail is breached.

Acceptance and recovery

Rehearse release, application rollback and database restore in staging. Publish a runbook with alert contacts. Keep incompatible schema removal out of the pilot; application rollback does not automatically undo data changes.

Task 2 - P4: Measurement and feedback

Measure / definition	Collection and owner	Review / proposed target
Lead time for change Median commit-to-production time	Git + deployment timestamps Dev lead	Weekly comparison to weeks 1-2 50% lower by week 12
Deployment frequency Production deployments per week	Release log Ops lead	Weekly pilot release from week 6 Only when release gates pass
Change failure rate Remediation-causing deployments / total	Linked incidents + release log Ops + Test leads	No increase against baseline Always show counts and sample size
Recovery time Incident start to service restoration	Alert + recovery timestamps Ops lead	Trend recovery weekly. Rollback drill ≤ 10 min. Report drills separately.

Sales reviews support themes weekly and puts user feedback into the shared backlog. Targets are proposals.

Task 2 - Resources and rollout gates

Proposed allocation | CEO + department heads

Pilot: two developers, one tester and one Ops engineer at 50% time; transformation lead at 20%. Sales product owner joins weekly prioritization. Confirm capacity and cloud budget in week 1.

Week 6 pilot gate | Ops + Test + Sales

Required checks pass; all-app schema tests pass; two staging release/rollback drills succeed; runbook and alert ownership are agreed. Then approve the first small production release.

Weeks 7-12 | Transformation lead

Collect two weeks of live pilot evidence, then onboard the second team in weeks 9-10 and the third in weeks 11-12 if gates pass. If reliability worsens, defer rollout and fix the constraint.

Task 3 - Evaluate the original plan

Existing item	Effect on experimentation / learning	Revision to the plan
P1 Shared board and squad	Positive: exposes queues and different perspectives. Negative: WIP can crowd out learning.	Reserve learning capacity; vary WIP in a documented experiment.
P2 Automated tests	Positive: quick evidence about defects. Negative: teams may treat green tests as complete assurance.	Pair Dev/Test; review escaped defects; add new regression cases.
P3 Release gates and standard tools	Positive: safe experiments and comparable logs. Negative: slow approvals or rigid standards block trials.	Delegate low-risk staging experiments; review gates and accept justified tool exceptions.
P4 Metrics dashboard	Positive: shows trends. Negative: rankings can encourage hiding problems.	Use team outcomes, include sample sizes, and discuss context without blame.

Culture suitability is unknown. Interviews and observed behavior must test assumption A3.

Task 3 - New learning actions

Action / owner / timing	Experiment or learning method	Evidence and acceptance
L1 Protected experiments Squad lead; from week 2	Reserve 10% of pilot time. Log a hypothesis, baseline, bounded staging test, success rule and stop rule.	One recorded experiment every 2 weeks; retain unsuccessful results and decisions.
L2 No-blame review Ops + Test; from week 3	Review incidents and near misses within 2 working days. Use a timeline and contributing system factors.	Publish review; assign actions and dates; close only after a test or runbook change.
L3 Shared learning Dev/Test leads; from week 3	Pair across roles; weekly 30-minute demo; shared wiki; monthly community meeting across all teams.	Show a lesson reused by another team; rotate demo ownership and maintain an index.

Resilience drills begin in staging only, with a named supervisor and an immediate stop/restore procedure.

Task 3 - Knowledge and culture

How knowledge is created

Queue maps reveal constraints; test failures reveal defects; release drills expose recovery gaps; user feedback reveals needs; experiments compare a hypothesis with observed results.

How knowledge becomes organizational

An individual records the finding -> Dev/Test/Ops review it -> publish a test, runbook or decision -> another team reuses it -> verify reuse at the next monthly review.

Culture change | CEO + department heads

Model no-blame reviews, welcome early risk reports and protect learning time. Reward shared delivery and reliability outcomes. Survey trust monthly and act on concerns; do not rank individuals by deployment counts.

Task 3 - Revised integrated timeline

Stage	Flow	Feedback	Learning (Third Way)
Weeks 1-2 Understand	Map queues; choose pilot; agree WIP and owners	Baseline delivery metrics; collect Sales feedback	Trust charter; reserve 10% learning time
Weeks 3-5 Build	Create pipeline; small commits; automate repeatable tests	Return failed checks to owners; instrument staging	Pair Dev/Test; log experiments; review near misses
Weeks 6-8 Validate	Gated small pilot releases; rehearsed rollback	Monitor live behavior; review metrics/support weekly	Staging resilience drill; publish findings; revise WIP and gates
Weeks 9-12 Scale	Onboard other teams only when pilot gates pass	Compare metrics across teams with shared definitions	Peer onboarding; reuse runbooks; share monthly

Revision: culture and protected learning now start before tooling and rollout. Scale remains conditional.

Complete plan - Handover and review

Execution pack | Transformation lead

Deliver the value-stream map, squad charter, backlog/WIP policy, CI configuration, test ownership, database migration rules, release/restore runbooks, dashboard and experiment log. Each document has an owner.

Week 12 review | CEO + all department leads

Compare pilot outcomes with the recorded baseline; show sample sizes and unresolved risks. Check that another team can execute a release and reuse a documented lesson. Decide to scale, adjust or extend the pilot.

Main risks and responses

Feature capacity falls: Sales reprioritizes. Database coupling persists: retain compatibility checks. Trust is weak: leaders change incentives and review behavior. Results are inconclusive: collect more evidence before broader rollout.

Sources and scope of this reference

[1] Case facts and required tasks

COMP2013, Lab 2: Understanding DevOps Through a Case Study, pp. 1-3. Submission: group PowerPoint or 3-minute video.

[2] Principles and classroom practices

COMP2013, Lecture 02: slides 24, 27-44 and 51-61. Used for Flow, Feedback, learning culture, CI/CD and delivery metrics. Layout follows the supplied DevOps template.

Assumptions and further reading

A1-A7, the 12-week schedule, staffing and targets are proposed. Business domain and baseline performance remain unknown. Recommended reading: The DevOps Handbook, chapters 1-3; not independently consulted for this deck.